

COGNIZANT

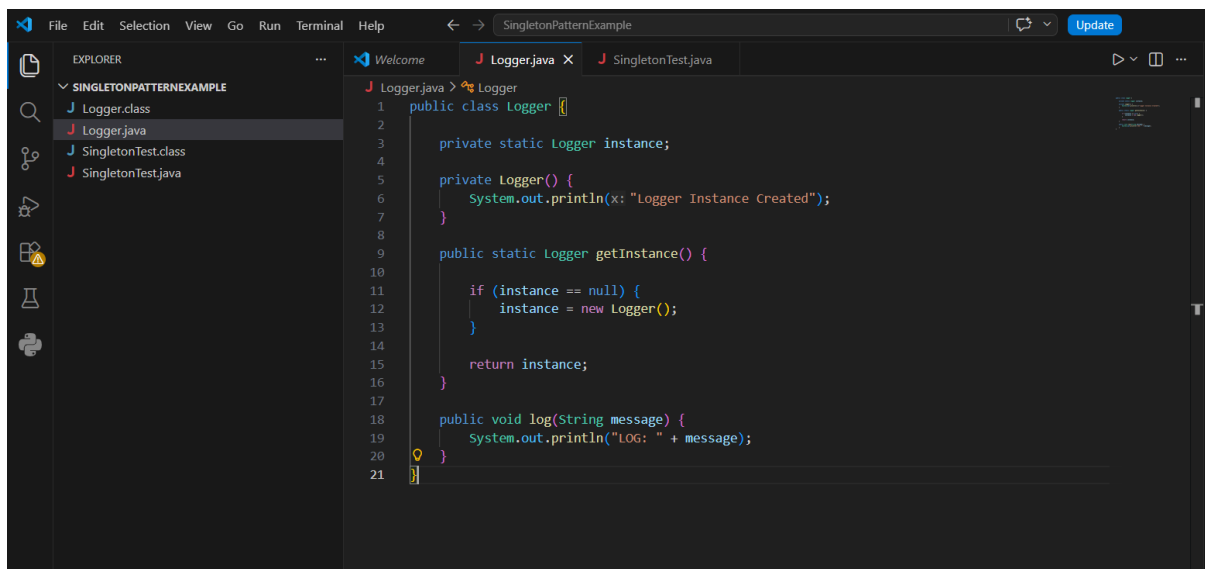
Mandatory Hands-on Exercises

-KHUSHI AHI

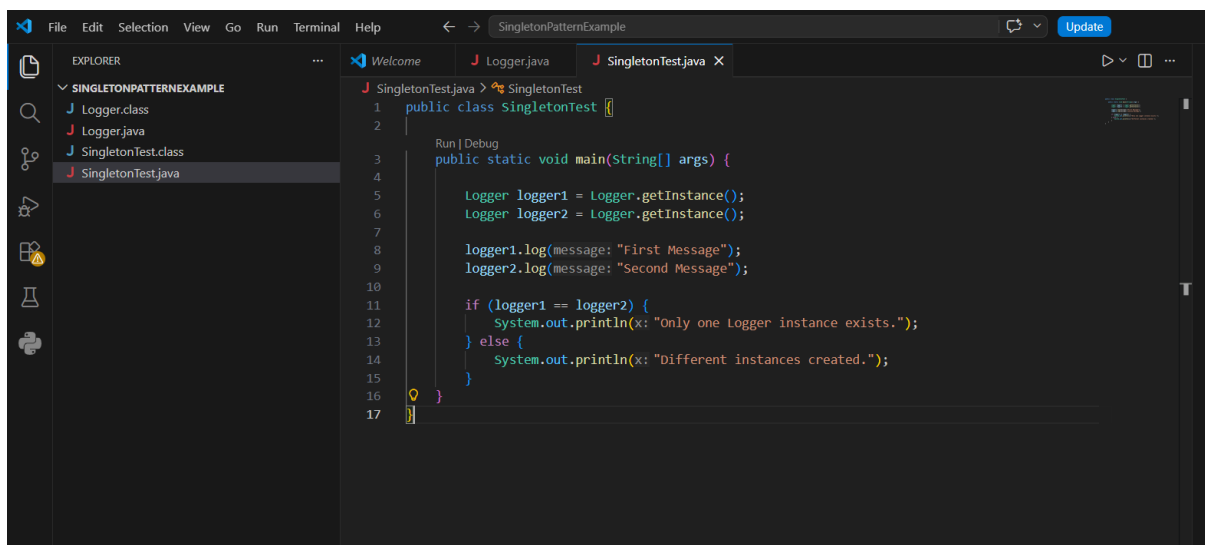
TOPIC: Design Patterns and Principles

Exercise 1: Implementing the Singleton Pattern

CODE



```
1 public class Logger {
2
3     private static Logger instance;
4
5     private Logger() {
6         System.out.println(x: "Logger Instance Created");
7     }
8
9     public static Logger getInstance() {
10
11         if (instance == null) {
12             instance = new Logger();
13         }
14
15         return instance;
16     }
17
18     public void log(String message) {
19         System.out.println("LOG: " + message);
20     }
21 }
```



```
1 public class SingletonTest {
2
3     public static void main(String[] args) {
4
5         Logger logger1 = Logger.getInstance();
6         Logger logger2 = Logger.getInstance();
7
8         logger1.log(message: "First Message");
9         logger2.log(message: "Second Message");
10
11         if (logger1 == logger2) {
12             System.out.println(x: "Only one Logger instance exists.");
13         } else {
14             System.out.println(x: "Different instances created.");
15         }
16     }
17 }
```

OUTPUT

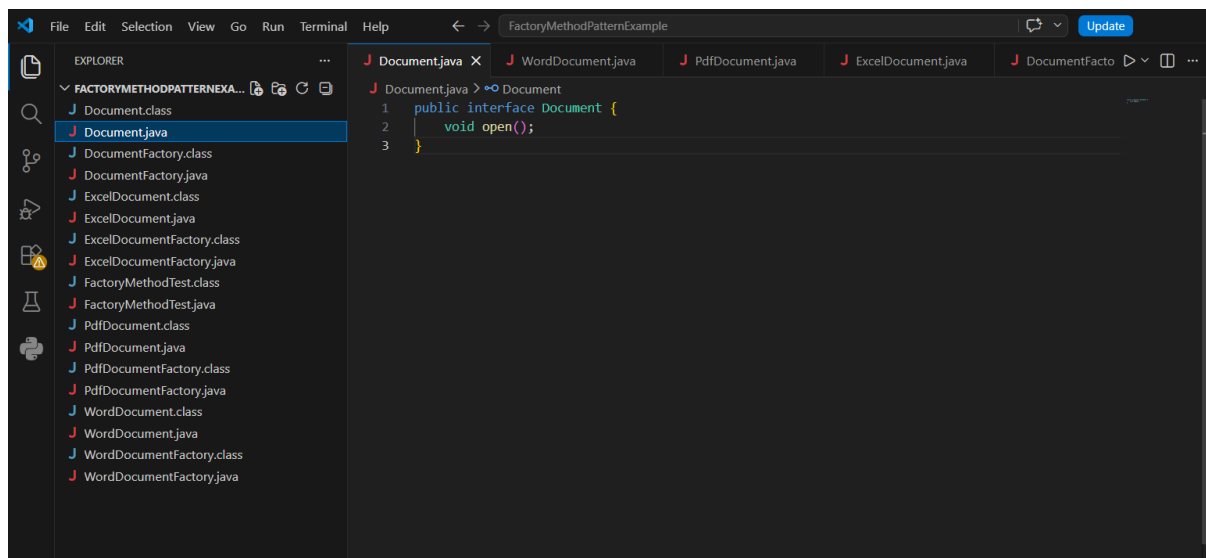
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(c) Microsoft Corporation. All rights reserved.

C:\Users\Khushi Ahi\OneDrive\Desktop\SingletonPatternExample>javac Logger.java SingletonTest.java

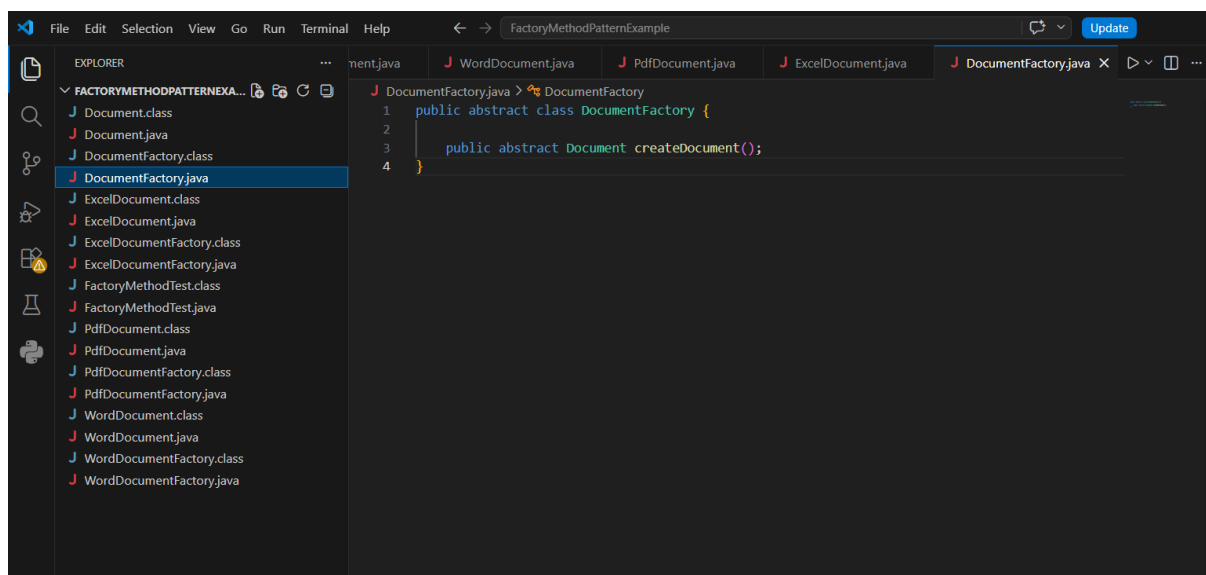
C:\Users\Khushi Ahi\OneDrive\Desktop\SingletonPatternExample>java SingletonTest
Logger Instance Created
LOG: First Message
LOG: Second Message
Only one Logger instance exists.
```

Exercise 2: Implementing the Factory Method Pattern

CODE



```
File Edit Selection View Go Run Terminal Help
FactoryMethodPatternExample
Document.java WordDocument.java PdfDocument.java ExcelDocument.java DocumentFacto
Document.java
1 public interface Document {
2     void open();
3 }
```



```
File Edit Selection View Go Run Terminal Help
FactoryMethodPatternExample
DocumentFactory.java WordDocument.java PdfDocument.java ExcelDocument.java DocumentFacto
DocumentFactory.java
1 public abstract class DocumentFactory {
2
3     public abstract Document createDocument();
4 }
```

This screenshot shows the Visual Studio Code editor with the 'FactoryMethodPatternExample' project. The Explorer sidebar on the left lists the project files, with 'ExcelDocument.java' selected. The main editor window displays the code for 'ExcelDocument.java', which implements the 'Document' interface. The code defines a public class 'ExcelDocument' that implements the 'Document' interface. It includes an '@Override' annotation for the 'open()' method, which prints 'Opening Excel Document' to the console.

```
1 public class ExcelDocument implements Document {
2
3     @Override
4     public void open() {
5         System.out.println(x: "Opening Excel Document");
6     }
7 }
```

This screenshot shows the Visual Studio Code editor with the 'FactoryMethodPatternExample' project. The Explorer sidebar on the left lists the project files, with 'ExcelDocumentFactory.java' selected. The main editor window displays the code for 'ExcelDocumentFactory.java', which implements the 'ExcelDocumentFactory' interface. The code defines a public class 'ExcelDocumentFactory' that extends the 'DocumentFactory' interface. It includes an '@Override' annotation for the 'createDocument()' method, which returns a new 'ExcelDocument' object.

```
1 public class ExcelDocumentFactory extends DocumentFactory {
2
3     @Override
4     public Document createDocument() {
5         return new ExcelDocument();
6     }
7 }
```

This screenshot shows the Visual Studio Code editor with the 'FactoryMethodPatternExample' project. The Explorer sidebar on the left lists the project files, with 'PdfDocument.java' selected. The main editor window displays the code for 'PdfDocument.java', which implements the 'PdfDocument' interface. The code defines a public class 'PdfDocument' that implements the 'Document' interface. It includes an '@Override' annotation for the 'open()' method, which prints 'Opening PDF Document' to the console.

```
1 public class PdfDocument implements Document {
2
3     @Override
4     public void open() {
5         System.out.println(x: "Opening PDF Document");
6     }
7 }
```

This screenshot shows an IDE window titled 'FactoryMethodPatternExample'. The Explorer on the left lists various files, with 'PdfDocumentFactory.java' selected. The main editor displays the code for 'PdfDocumentFactory', which extends 'DocumentFactory'. It contains an '@Override' method 'createDocument()' that returns a new 'PdfDocument()' object.

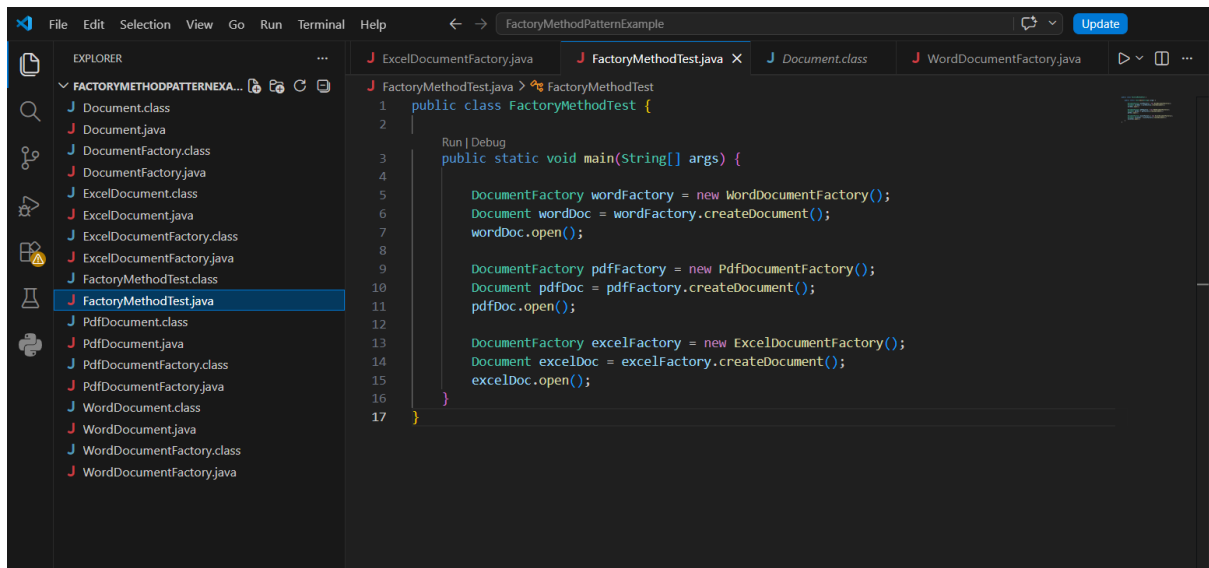
```
1 public class PdfDocumentFactory extends DocumentFactory {
2
3     @Override
4     public Document createDocument() {
5         return new PdfDocument();
6     }
7 }
```

This screenshot shows the same IDE window with 'WordDocument.java' selected in the Explorer. The main editor displays the code for 'WordDocument', which implements the 'Document' interface. It contains an '@Override' method 'open()' that prints 'Opening Word Document' to the console.

```
1 public class WordDocument implements Document {
2
3     @Override
4     public void open() {
5         System.out.println(x: "Opening Word Document");
6     }
7 }
```

This screenshot shows the same IDE window with 'WordDocumentFactory.java' selected in the Explorer. The main editor displays the code for 'WordDocumentFactory', which extends 'DocumentFactory'. It contains an '@Override' method 'createDocument()' that returns a new 'WordDocument()' object.

```
1 public class WordDocumentFactory extends DocumentFactory {
2
3     @Override
4     public Document createDocument() {
5         return new WordDocument();
6     }
7 }
```



OUTPUT:

